

这是一个关于**二叉堆（Binary Heaps）和优先队列（Priority Queues）**的讲座总结。

这份 PPT 主要介绍了优先队列的概念、为什么我们需要它、二叉堆的结构特性、核心操作（插入、删除最小）的实现细节以及时间复杂度分析。

以下是详细的总结：

第6讲：二叉堆与优先队列 (Heaps & Priority Queues)

1. 为什么需要优先队列？ (Motivation)

在计算机科学中，我们经常需要**快速找到最小（或最大）的元素**，而不是像普通队列那样遵循“先进先出”（FIFO）的规则。

- 应用场景：
 - **操作系统调度**：根据优先级调度任务，而不是按提交顺序。
 - **事件模拟**：银行客户到达或离开，按事件发生的时间顺序处理。
 - **查找最值**：寻找成绩最高的学生、工资最高的员工等。

2. 优先队列 ADT (Priority Queue ADT)

优先队列是一种抽象数据类型，主要支持以下操作:

- 1. **FindMin**（查找最小值）
- 2. **DeleteMin**（删除最小值）
- 3. **Insert**（插入新元素）

现有数据结构的局限性对比

数据结构	Insert 复杂度	FindMin 复杂度	缺点
链表 (排序)	$O(N)$	$O(1)$	插入太慢
链表 (未排序)	$O(1)$	$O(N)$	查找太慢
二叉搜索树 (BST)	$O(\log N)$	$O(\log N)$	性能不错，但属于“大材小用”，我们只需要最小值，不需要全部有序
哈希表	$O(1)$	困难	无法有效支持查找最值

我们的目标：

- **FindMin:** $O(1)$
- **Insert:** $O(\log N)$
- **DeleteMin:** $O(\log N)$

3. 二叉堆 (Binary Heaps)

二叉堆是实现优先队列的常见方式。它是一种二叉树，但**不是**二叉搜索树 (BST)。它必须满足两个核心性质：

A. 结构性性质 (Structure Property)

- **完全二叉树 (Complete Binary Tree):**
 - 树被完全填满，除了底层。
 - 底层元素必须**从左到右**填充，不能有空隙。
 - **优点:** 因为是完全二叉树，树的高度严格控制在 $\lfloor \log N \rfloor$ 。

B. 堆序性质 (Heap Order Property)

- 对于最小堆 (Min-Heap)：父节点的值总是小于或等于其子节点的值。
- 因此，**根节点 (Root)** 总是整棵树中最小的元素。
- **注意:** 堆只保证路径有序，不保证左右子树之间的相对顺序 (这与 BST 不同)。

4. 堆的数组实现 (Array Implementation)

由于二叉堆是完全二叉树，我们可以直接使用**数组**来存储，不需要指针，这样更节省空间且访问速度快。

索引计算规则

假设根节点存储在数组索引 $A[1]$ 的位置 ($A[0]$ 通常留空或存放特殊值)：

- 对于任意节点 i ：
 - **左子节点 (Left Child):** $2 * i$
 - **右子节点 (Right Child):** $2 * i + 1$
 - **父节点 (Parent):** $i / 2$ (向下取整)

5. 核心操作详解

5.1 查找最小值 (FindMin)

- **操作：** 直接返回数组的第一个元素 $A[1]$ （根节点）。
- **复杂度：** $O(1)$ 。

5.2 删除最小值 (DeleteMin)

当删除根节点（最小值）后，树会出现一个“空洞”(Hole)。为了保持**结构性质**（完全二叉树），我们必须把数组的**最后一个元素**移到根的位置，但这通常会破坏**堆序性质**。

- **算法流程（下滤 Percolate Down）：**
 - i. 移除根节点 $A[1]$ 。
 - ii. 将堆中最后一个元素 x 暂时放入根节点位置。
 - iii. **比较** x 与其两个子节点。
 - iv. 如果子节点比 x 小，将**较小的那个子节点**上移填补空洞，空洞下移。
 - v. 重复此过程，直到 x 小于两个子节点，或者空洞已经落到底层。
- **复杂度：** 与树高成正比，即 $O(\log N)$ 。

5.3 插入 (Insert)

为了保持**结构性质**，新元素必须放在下一个可用的位置（即数组的末尾）。但这可能会破坏**堆序性质**（新元素可能比父节点小）。

- **算法流程（上滤 Percolate Up）：**
 - i. 在数组末尾创建一个空洞。
 - ii. 比较新元素 x 与其**父节点**。
 - iii. 如果 x 小于父节点，则将父节点移下来（填补空洞），空洞上移。
 - iv. 重复此过程，直到 x 大于父节点，或者空洞到达根位置。
- **复杂度：** 最坏情况 $O(\log N)$ 。平均情况非常快，经常是常数时间。

6. 建堆 (BuildHeap)

如果我们有一组无序的数据，如何构建一个堆？

- **方法 1 (Naive)：** 执行 N 次 Insert 操作。
 - 最坏复杂度： $O(N \log N)$ 。
- **方法 2 (Linear Time)：** 从最后一个非叶子节点开始，依次向前进行 **下滤 (Percolate Down)** 操作。
 - 代码逻辑：`for (i = N/2; i > 0; i--) percolateDown(i);`
 - **复杂度：** 经过数学推导，这种方法的复杂度是 $O(N)$ （线性时间）。这是因为大部分节点在树的底层，下滤的高度很小。

7. 总结与分析 (Analysis)

特性	说明
空间复杂度	$O(N)$ (数组存储)
FindMin 时间	$O(1)$
Insert 时间	$O(\log N)$
DeleteMin 时间	$O(\log N)$
BuildHeap 时间	$O(N)$

课后练习： 练习 6.2, 6.3, 6.4 。